Informalizing Lean Mathlib

Mathlib in English, French, Czech, German and Swedish with Informath, built with Claude Code

Aarne Ranta

23 October 2026

Lean and Mathlib

- Lean 4: a proof assistant and functional programming language based on dependent type theory (de Moura et al.; now the Lean FRO).
- Mathlib: Lean's community library of formalized mathematics, from algebra and number theory to analysis, topology and category theory.
- At the commit used here (`c31114ff81`, 21 Oct 2026): 8,650 files, 2.3 million lines, about 194,000 theorems and 48,000 definitions, structures and classes.
- Statements are read by Lean's elaborator: type classes, implicit arguments, coercions and notation are resolved there, not in the source text.

```
/-- `Nat.Prime p` means that `p` is a prime
    number,
  that is, a natural number at least 2 whose
     only
  divisors are `p` and `1`. -/
@[pp_nodot, wikidata Q49008, oeis A000040]
def Prime (p : ℕ) :=
  Irreducible p

theorem exists_prime_and_dvd {n : ℕ} (hn : n
    ≠ 1) :
    ∃ p, Prime p ∧ p ∣ n :=
  ⟨minFac n, minFac_prime hn, minFac_dvd ⟩_
```

Mathlib/Data/Nat/Prime/Defs.lean

The problem: Mathlib is formal

What informal text there is

- Docstrings, written by hand. Batteries' `docBlame` linter requires them for definitions; `docBlameThm`, the same for theorems, is disabled. In our checkout, 83% of definitions, structures and classes have one, but only 15% of theorems and lemmas.
- Module docstrings summarizing each file, and the HTML documentation generated by doc-gen4, which shows the formal statements with the docstrings.
- LLM informalizations made for search and for training: LeanSearch (Gao et al., EMNLP Findings 2024) and the Herald dataset (Gao et al., ICLR 2025), checked by back-translation.

What is missing

- Readings of all statements that are faithful by construction, reproducible, and in several languages.
- Proofs in natural language: none of the above covers them.

GF and Informath

GF, Grammatical Framework

- An abstract syntax (trees) with concrete syntaxes (linearizations) for many languages.
- The Resource Grammar Library takes care of morphology and syntax: agreement, word order, inflection.

Informath (2024–, 932 commits)

- Translates between Dedukti and natural language, in both directions, with LaTeX formulas.
- A Dedukti dialect: `Set`, `Elem`, `Prop`, `Proof`, and a base of arithmetic and logic.
- Symbol tables (`.dkgf`) give each constant its verbal and symbolic forms.

From Dedukti to text

1. Annotate the constants with the symbol tables.
2. Translate to MathCore, a minimal GF syntax.
3. Generate the variants: verbal or symbolic, aggregated, in-situ quantifiers...
4. Linearize them in English, French, Swedish, ...or as LaTeX formulas.
5. Rank them, and pick the best.

Proof texts (`-proof-text`): numbered lines, each justified by a rule or a lemma, as in godement-index.

Building mathlib-informath

Two days, thirteen prompts

	prompt (abridged)	outcome	commits
1	build an informalization system for Mathlib	exporter, Nat primes	`c46c6c3`
2	build mathlib.dkgf and apply it	17 entries: 69 → 7 unworded	
3	an HTML page with Lean, Dedukti, MathJax, GitHub links	the page, then GitHub Pages	
4	split it into subfiles; Factorization next	322 statements on 2 pages	`3e8bc29`
5	keep the definientia, show the proofs	84 proof texts; Informath fix	`5e78612` `1658da6`
6	natural language from the sources	docstrings and names; `Mathlib.gf`	`8244cae`
7	macros in red: MathJax issue?	a builder bug; $\mathsf{ord}_p(n)$	`6b7383c`
8	the Algebra and Order.Defs modules	705 statements on 4 pages	`976bbd0`
9	add French	a third row, an index	`823155a`
10	add Czech, hidden with French; buttons at each entry	a fourth row	`cacc011` `0c9c416`
11	aggregate hypotheses (in Informath); rebuild the pages	503 readings change	`afd0d9d`
12	add German	a fifth row, with a time limit	`a53c372`
13	proofs that use tactics, as proof units	25 tactic proofs (English)	`9822ffa`

21–22 October 2026: 13 commits in mathlib-informath (besides these slides) and 8 in informath (`4f31f55`, `ceb9b85`, `7773194`, `778e998`, `0da10d2`, `5907eec`, `a8b863f`, `4bcff16`), all made by Claude when asked, and pushed by me.

1. The plan: three decisions

> "you can see ../mathlib4, right? It is Leans famous Mathlib. I would like to build an informalization system for it using Informath."

Claude surveyed Mathlib, Informath's Lean support and lean2dk (Deducteam's faithful Lean-to-Dedukti translator), then asked three questions:

1. Input: own Lean exporter (recommended), lean2dk, or parsing the Lean source. lean2dk encodes the whole type theory, which is the opposite of what Informath reads; the source text lacks what the elaborator adds.
2. First slice: Algebra/Group/Basic, Nat primes, or orders and lattices.
3. Setup: a new project, with Mathlib's build cache downloaded (7 GB in two minutes, instead of hours of compiling).

The answers are in green. The first run read all 130 statements of the five prime modules back into English, 61 of them already fully worded.

The exporter: from Lean's environment to Dedukti

`Export.lean` (now 500 lines) runs over Mathlib's elaborated environment:

- binders become `Elem`, `Proof` or `Set` hypotheses;
- implicit and instance arguments are dropped from applications;
- casts and coercions are the identity, numerals Informath's digits;
- `mapping.tsv` maps Lean's notions to Informath's base: `Nat.Prime`→`prime`, `Dvd.dvd`→`divide`;
- per module: `.dk`, `.json` (docstrings, lines), `-deps.dk`.

```
theorem Prime.eq_one_or_self_of_dvd {p : ℕ} (pp
      : p.Prime)
    (m : ℕ) (hm : m ∣ p) : m = 1 ∨ m = p
```

```
Nat__Prime__eq_one_or_self_of_dvd : (p : Elem
     Nat) ->
  (pp : Proof (prime p)) -> (m : Elem Nat) ->
  (hm : Proof (divide m p)) -> Proof (or (Eq m
     1) (Eq m p)).
```

Let p be a natural number. Assume that p is prime. Let m be a natural number. Assume that m divides p. Then m = 1 or m = p.

$\mathrm{p} \in \mathbb{N}, \mathrm{prime}(\mathrm{p}), \mathrm{m} \in \mathbb{N}, \mathrm{m} \mid \mathrm{p} \vdash \mathrm{m} = 1 \vee \mathrm{m} = \mathrm{p}$

2. A symbol table for Mathlib's constants

"build mathlib.dkgf and apply it"

- 17 entries, worded after Mathlib's docstrings where the grammar had the words, checked against the exported types: `Nat.minFac` "the minimal prime factor of #1", `Nat.Coprime` "#1 and #2 are coprime" | #1 ⊥ #2.
- Readings with an unworded constant: 69 → 7 of 130.

Writing the table exposed three exporter bugs

- Prop-valued constants were typed `Set`.
- Lean's `Set` clashed with Informath's: it now maps to `powerset`.
- Definitions were declared with implicit arguments that the applications drop, so `#k` counted the wrong arguments.

And Informath issues, reported but not fixed

`$` inside a quoted verbal form splits a table entry; a proposition inside a macro argument loses its notation; "does not have a upper bound".

3. The page, and GitHub Pages

> “produce a mathlib-informath.html file, which shows for each statement the original lean, the dedukti translation, and the informalization in mathjax latex, as well as a link to the original lean source in mathlib github”

- Each entry: name, kind, docstring, the Lean source up to its `:=`, the Dedukti, English, the formula, and a link to the line at Mathlib’s exact commit.
- The exporter now records source positions and keeps the source order. Lean-generated declarations, which have no position, are left out.
- Checked in headless Chrome: MathJax renders the readings.

> “will it be hosted in github?”

Not yet: the project had no remote. Claude explained the two steps, a repository and GitHub Pages, but left both to me, as they publish under my name. I created `aarneranta/mathlib-informath`; Claude committed `c46c6c3`; I pushed.

4. More of Mathlib: Factorization, and a page per part

> "Do more of Mathlib. But now you should split the html document to a main file, which contains a table of contents with links to subfiles of reasonable size. ... Factorization is perhaps the next example?"

- One page per directory; the table of contents forwards old links (`#Nat.Prime.two_le`) to the right page, since the page is public.
- Coercions to functions: `n.factorization p` becomes one constant `Nat__factorization__app n p`.
- Binders ($\prod_{x \in s} f(x)$, $\{x \in s \mid P\}$): the exporter eta-expands the function arguments, and raw macros take the bound variable as a parameter, as Informath's `\Summa`.
- Type classes become predicates: `[CommMonoid M]` reads "Assume that M is a commutative monoid".
- About 30 new table entries; deprecated aliases left out; a `-force-symbolic` fallback for statements with no reading.

Result: 322 statements on two pages, 267 fully worded (`3e8bc29`).

5. Definientia and proofs

> "I don't think we need to cut judgements at := if the definiens is an explicit expression directly translatable to Dedukti. Keep it in those cases, and if the value type is Proof X, show the proof in the same way as in godement-index."

- Only term definientia (not `by` tactics, not equations). Theorems' proofs needed `value? (allowOpaque := true)`.
- Proofs go into the natural deduction rules of `nd.dk`: λs proving $\to$ or $\forall$ become `ifI`/`forallI`, applied hypotheses `ifE`/`forallE`.
- Lemmas keep their arguments in proofs, except type class instances.
- An Informath fix (`4f31f55`): `-proof-text` crashed on a λ typed by a proof of an implication.

```
theorem exists_prime_and_dvd {n : ℕ} (hn : n
    ≠ 1) :
  ∃ p, Prime p ∧ p ∣ n :=
⟨minFac n, minFac_prime hn, minFac_dvd ⟨_
```

1.	Let n be an element of $\mathbb{N}$.	
2.	Assume that $n \neq 1$.	
3.	minFac(n) is prime.	`minFac_prime` 2
4.	minFac(n) $\mid$ n.	`minFac_dvd`
5.	minFac(n) is prime and minFac(n) $\mid$ n.	`andI` 3, 4
6.	There exists an element p of $\mathbb{N}$, such that p is prime and p $\mid$ n.	`existsI` 5

6. Natural language from the sources

> "find more natural language expressions instead of verbatim Lean constants. … Consider for instance primeFactorsList(n) … Could this information be found somewhere in the available sources?"

Three sources, as Claude found them:

- docstrings, in fixed patterns: "`f n` is the X of n", "Returns the X…";
- names, by Mathlib's naming conventions: `IsUnit` → "#1 is a unit";
- Wikidata attributes, but only on a few declarations.

`suggest-entries.py` makes candidates from the first two and keeps those that `-parse-example` parses.

> "if you need new words, add them to Mathlib.gf"

- 31 reviewed entries: "the smallest prime factor of #1", "the set of prime factors of #1".
- `Mathlib.gf`: "smallest", "descending", "bounded above", "set of divisors"… (`ceb9b85`).
- English is read from a table without pseudo-notations such as primeFactors(n), which the ranking preferred.

Factorization readings with a raw Lean name: 94 → 8 of 194.

7. Red macros

> "In Factorization.html, some macros are rendered in red and not expanded. For instance, \NatfactorizationappMACRoI. Is this a MathJax Issue or something easy to fix?"

- Not MathJax: Claude's own bug from step 6. Macros are numbered by their position in the table, and the English table, with fewer LaTeX forms, numbered them otherwise. 37 macros were undefined.
- The fix: the pages define the macros of both tables, with a check for clashing names (none).
- The screenshot check found another problem: the multiplicity of p in n and the p-adic valuation were written $\nu_{\mathrm{p}}(\mathrm{n})$ and $\mathrm{v}_{\mathrm{p}}(\mathrm{n})$. Claude asked; I chose to commit both changes, and the multiplicity is now $\mathsf{ord}_{\mathrm{p}}(\mathrm{n})$ (`6b7383c`).

8. Down the imports: Algebra and Order

> "I can see that already Prime.Defs imports some Algebra modules: Group, Prime, etc. Can you run informalization with those Algebra modules and also Order.Defs modules?"

- The five `Order.Defs` modules (354 statements) and two Algebra modules; `GroupWithZero.Nat` has only instances and was left out.
- `# page: X` lines in `modules.txt` put small modules on one page.
- About 40 entries for order, algebra and relation classes: "#1 is partially ordered", "#2 is a strict weak order", "the supremum of #1 and #2" | #1 ⊔ #2; four more words in `Mathlib.gf`.
- `Decidable` instance hypotheses are computational, so the exporter omits them.
- "r holds for a and b", a relation variable, now counts as worded.
- The slow proof texts (`sup_assoc`) got a time limit.

Result: 705 statements on four pages, 574 fully worded (`976bbd0`).

9. French

"Now, let us add French! Can you figure out what is needed?"

Claude's findings:

1. the full grammar was six weeks old;
2. French proof texts were empty: `ProofUnitsFre` lacked the two functions that proof lines are made of;
3. 27 words lacked French, in 5 modules;
4. the page builder needed a French row;
5. grammar flaws, for later.

"Do 2-4 and add French as a third row for the time being"

`le_antisymm`:

Let x be a set. Assume that x is partially ordered. Let a and b be elements of x. Assume that a $\leq$ b. Assume that b $\leq$ a. Then a = b.

Soit x un ensemble. Supposons que x est partiellement ordonné. Soient a et b des éléments de x. Supposons que a $\leq$ b. Supposons que b $\leq$ a. Alors a = b.

9. French: two interruptions

The first build took minutes: every French call loads the full grammar (2.8 s), and the proofs were read one at a time.

> "It is better to build an index that maps Dedukti to French and English and LaTeX in one go, so that the grammar is only loaded once. Then use this index to populate the html."

- Informath (`778e998`): `-languages=Eng,Fre` for `-json` and `-proof-text`, and `-no-back-to-dedukti`, as the back-translation crashed on proofs.
- `build-index.py` → `index.json` in 27 s, and `build-html.py` renders it in 0.06 s.
- Keyed by Dedukti names, the index revealed that the old pages had put some readings on the wrong declarations.

> "don't add german yet, just French and English"

Claude had rebuilt the grammar with German, to keep a regression test running; the full grammar now has English and French only (`823155a`).

10. Czech

> “Now that the grammar with 7 languages is in place, why not add Czech to the html file - to begin with. Please hide French and Czech by default but make them easy to click visible.”

- The Czech grammar had its user extensions switched off: `HottWordsCze` was a stub with two Swedish words, and did not compile. Now in Czech, and on (`0da10d2`).
- No `ProofUnitsCze`: proof texts would be empty, as French ones were.
- The same 27 words as for French: `MathlibCze` (nejmení, shora omezený, mnoina dlitel . . .) and partial modules. Unreviewed.
- `build-index.py`: RunInformath loses its buffered output when it stops on an error, which cut off the proof before a failing one; a proof now counts as failed only if it fails alone.
- A check box for each hidden language above the entries, remembered in the browser.
  > “add Czech and French links locally at each judgement, where they are easier to find.”

A French and a Czech button in each entry’s header line (`0c9c416`).

11. Aggregated hypotheses

> "optimize hypotheses by using AdjKindHypo and SuchThatHypo … even sorting the hypotheses so that this becomes more possible … whenever the type dependencies are obeyed."

`aggregateHypos` in Informath (`5907eec`):

1. one hypothesis per variable;
2. a variable moves back next to the last one of its kind, if its kind depends on nothing in between;
3. variables of a kind regrouped;
4. the propositions about them attached: an adjective each, the same, or else "such that".

The old rule had never applied: unnamed hypotheses arrive with generated labels.

503 of 705 English readings change (`afd0d9d`):

- 105 adjectives: "Let p and q be prime natural numbers."
- 315 "such that": "Let p and m be natural numbers such that p is prime and m divides p" — m was declared after "Assume that p is prime".
- 83 class hypotheses: "Let x be a set such that x is a join semilattice."

Remaining: "a set such that x is a join semilattice" could be "a join semilattice"; "such that" after "a function from … to sets" seems to be about the sets; mixed wordings of a constant keep variables apart.

12. German, and a time explosion

"I know that this can be heavy, but can you add German to Mathlib-informath?"

- German had the least: its user extensions were Naproche only, and its Wikidata lexicon partial. The table needed 67 German words, 40 of them general (Kardinalität, Bild, Vereinigung . . .); new `ProofUnitsGer`, `MathlibGer` and partial modules. Unreviewed.
- Then the batch call did not finish. GF's German linearization is exponential in nested noun phrases: "the minimum of the minimum of a and b and c" — depth 2 in 0.3 s, depth 3 in 35 s; English is instant.

"aggressive time limit per statement. I will try to fix German rgl."

- German is read apart, in chunks of 12 with 1 s per statement; a chunk over the limit is split in halves down to the slow statements, which get no German, marked on the page.
- 48 statements left out — the lattice laws (`sup_assoc` . . .) and the `ordCompl` lemmas; all 240 proofs have German. The index takes 15 minutes instead of one.
- German flaws seen: "elemente", "Dann $2 \leq p$" (without gilt), "eine prime natürliche Zahl" for Primzahl.

13. Tactic proofs

> "Now I'm interested in adding proofs that use tactics. This might be doable by ignoring proof expressions in Dedukti and going directly to lists of proof units."

242 of our 620 theorems are proved by tactics, mostly `rw`, `simp`, `exact`; the scripts alone say little. Claude asked three questions: Lean's goals, a new mode `-tactic-proof`, Nat.Prime.Defs in English first.

- `export --tactics`: the module elaborated again (5 s), each tactic of a sequence with its goals before and after, in Dedukti.
- `tactic-units.py`: units from comparing the goals — assumed, obtained, stated, closed, "it is enough to show", cases.
- `RunInformath -tactic-proof`: the units through the symbol tables and the NLG, written out as by `-proof-text`.

```
obtain ⟨n, ⟨hn := hm
have := pp.isUnit_or_isUnit hn
rw [Nat.isUnit_iff, Nat.isUnit_iff] at this
apply Or.imp_right _ this
rintro rfl
rw [hn, mul_one]
```

1. Let n be an element of $\mathbb{N}$ such that $p = m \times n$.
2. m is a unit or n is a unit.
3. $m = 1$ or $n = 1$.
4. It is enough to show that if $n = 1$, then $m = p$.
5. Assume that $n = 1$.
6. $m = p$.

Fixed on the way: `rintro rfl` assumed the substituted hypotheses; `by_cases` lost its case hypotheses.

The rebuild found that the RGL, changed in my German work, no longer compiles Czech.

14. Czech again, and optimized grammars

"can you fix the Czech grammar ?"

- The RGL's Czech had changed: `NP` with clitics and pro-drop, 32 agreement values, new `mkV2`, `mkVS`. Informath's `[ArgKind]` then had 536,870,912 concrete functions; an `ArgKindNP` without the new fields brings it to 1,048,576. "Pedpokládejme" and "vechna" came for free.
- Then my `-optimize-pgf` for smaller grammars broke all proof texts: Index (-1) out of range. Claude bisected it to the optimization; I knew why: units are not reachable from the start category, so I added `UnitJmt`.
- Claude added `ProofUnits` for Swedish, Czech and German (the units of tactic proofs), with "dass p prim ist" (verb last), and the Swedish words of the Mathlib pages: Swedish became the fifth language.

15. A symbol table for all of Mathlib

"Build a symbol table for all of Mathlib's definitions. Use information from docstrings and also try to parse camel script to multiword expressions. Add words to Informath lexicon if needed. Start by doing this just for English"

- `export --definitions`: all of Mathlib imported once (20 s), 56,867 definitions, structures, classes, fields; 38,624 after leaving out tactics, notations, auxiliaries.
- `definition-table.py`: candidates from the docstring ("The cofinality of an ordinal is ...") and the name (`minFac`: "the minimum factor of #1"), Mathlib's abbreviations spelled out; all parsed in one call (30 s); the first whose category and arity fit is the entry.
- New words for the grammar from DictEng (categories, plurals): about 500.
- 70% of the definitions have a wording (26,154): functions 22,641 of 31,858, types 1,792 of 2,457, predicates 1,721 of 4,309.

16. Readings that are right, or none

Rounds of sampling found the classes of errors:

- `AddMonoidHom`: "sum monoid homomorphism" → "additive monoid homomorphism"; `MulLeftMono` is about monotonicity, not monomorphisms.
- `Bicone.ofLimitCone`: "the limit cone of #1" → "the bicone of #1"; `pointOfClosedPoint` keeps its head, `PartitionOfUnity` its "of"; `compRight` is the right composition.
- Extra explicit arguments (a category, a topology): readings name the main ones, as `Eqt`'s "#2 is equal to #3" does.
- But `IsRelUpperSet s P` became "P is a relation upper set". A wrong reading is worse than a symbolic one: a predicate of two objects is now binary, "#1 is a relative upper set with respect to #2", its subject the argument the docstring does not name after "relative to", "for"; without such a clue, no reading (predicates 2,531 → 1,721).

The pages gain 14 fully worded statements: `UpperSet.carrier` is "the carrier of self".

17. The words in the other languages

> "As for lexicon, you can also try ../wordnet. The abstract syntax file gives hints about what domain each word sense is used on."

- GF's WordNet: 117 of 789 words, 13 by a sense tagged mathematics, and even those mixed: Sechseck, variance right; minus as Weg, triplet as trois; substructure as Infrastruktur by its gloss. Mathlib's jargon (cocone, hypercover, sheafification) is not there.
- Instead `translations.tsv`: one line per word with French, German, Swedish and Czech forms (gender, plural), written by Claude, to be reviewed; an empty cell where unsure. 473, 465, 441, 441 of 485 words.
- Where a language still lacks a word, `build-index.py` keeps the reading made without the generated table: 26 declarations, all with readings again.
- On the way: 300 junk words of docstrings out of the lexicon (because, banana, simproc).

18. A stand-alone informalizer

> "Write a stand-alone program that informalizes any piece of Lean code read from standard-in or from a file. ... Claude would no more be needed when running this program."

(`standalone.md`)

- `export --file`: any Lean file elaborated (about 35 s, mostly loading Mathlib), its declarations and tactic steps exported.
- `mathlib-informath`: the index code of the pages for one file; plain text, `-html`, `-pdf`; `-languages`, `-table` for one's own entries; `-check`, `-install`.
- Later: a web server keeping Lean loaded, and VS Code.

```
$ mathlib-informath Example.lean
prime_ne_one (theorem)
  English: Let $p$ be a prime natural number.
           Then $p \neq 1$.
  Proof:
    1. Assume that $p = 1$.     [intro h]
    2. $1$ is prime.            [rw [h] at hp]
    3. We have a contradiction.
                    [exact Nat.not_prime_one hp
     ]
```

19. All definitions on a page

"I would like to have a browsable page with the definitions."

- `definitions.html`: all 38,690 definitions with their wordings, 26,215 of them worded; the arguments by their names ("the cofinality of o"); the docstring's first sentence; a link to Mathlib's documentation.
- Search by name, wording and docstring; filters by part of Mathlib, by kind, and by where the wording comes from (by hand, docstring, name). The data is a 7 MB JSON file read by the page.
- Linked from the main page; the Dedukti rows of the entries are now folded, as the proofs are, as I asked: "It is only interesting for debugging."

20. All definitions read in English

> "I would like to have one more thing in definitions.html: the informalizations of each definition. The docstrings are not needed"

- `export --definitions` also writes the definientia (25,494); all 38,690 definitions read by the pages' code, 8 RunInformath processes, 1 s per definition: half an hour.
- 38,279 (99%) with a reading, 405 over the time limit; about 12,300 (32%) with no constant shown symbolically.
- A sample of 99 first, as I asked; but it loaded its data by `fetch`, which a browser refuses for a file:

  > "I couldn't see anything. Why not build a static html similar to mathlib-informath?"

- Now 111 static pages of at most 400 definitions, and a search that jumps to them.
- "Let V be a set. Then V is thin, if for all elements a and b of V, the set of homomorphisms of a and b is a subsingleton." (`Quiver.IsThin`)
- The other languages: possible on demand, same pipeline; not made yet.

21. Camel case to words of the lexicon

"find English wordings for the constants now left with original Lean camel script. Can you match them with what is available in the Informath lexicon and find suggestions, cross-checking with already available symbol table entries for Mathlib?"

- 12,816 definitions without a wording: 4,472 with a word the grammar lacks, 5,249 whose words it has but not as nouns (`restrict`), 3,095 with no candidate.
- `lexicon-repair.py`: verbs by their nominalizations in the lexicon (`restrictScalars`: "the scalar restriction of #1"), abbreviations by the word that the docstrings of the definitions named with them use (`denom`: denominator), proper names spelled (`kronecker`: Kronecker), `desc`: "the induced morphism".
- Copying the entries of other constants with the same name was wrong: `Cotrident.ofCocone` is no "pushout cocone".
- First 30% wrong, as in the first table, all in long names ("the domain homomorphism restriction"); restricted to names of at most two words: 887 more entries, 135 replacements to review; a sample of 60: 40 good, 14 acceptable, 6 wrong (10%).
- The setup had to be rebuilt first: Mathlib's cache (7 GB), the export, Informath's grammar and `RunInformath`.

22. Formulas on the definitions pages

"In definition pages, I would like to see the logical formula produced by Informath together with the English statement."

- Each definition is read also by `-to-symbolic-latex`, with the same time limit; a call must give one formula per definition, or it is split.
- 38,460 of 38,667 definitions with a formula, 38,424 with an English reading; both in 27 minutes with 8 processes.
- `Quiver.IsThin`: "Let V be a set. Then V is thin, if for all elements a and b of V, the set of homomorphisms of a and b is a subsingleton."

  $\mathrm{V} \in \mathrm{Set} \vdash \mathsf{IsThin}(\mathrm{V}) :\Leftrightarrow \forall \mathrm{a} \in \mathrm{V}.\ \forall \mathrm{b} \in \mathrm{V}.\ \mathrm{Subsingleton}(\mathsf{Hom}(\mathrm{a}, \mathrm{b}))$

The result

Fully worded statements:

page	statements	English	French	Czech	German	Swedish
Mathlib.Order.Defs	354	291	289	289	248	289
Mathlib.Algebra	29	29	29	29	28	28
Mathlib.Data.Nat.Prime	128	125	125	125	125	125
Mathlib.Data.Nat.Factorization	194	143	143	143	132	142
all	705	588	586	586	533	584

- 264 declarations with their definiens; 240 proof texts in each language, out of 242 term proofs; 51 statements without German (time limit).
- 25 tactic proofs in all five languages (`Nat.Prime.Defs`), out of 242.
- 26,741 of Mathlib's 38,667 definitions worded; 38,424 read in English, 38,460 as formulas (`definitions.html`).
- Every statement has a formula; 31 need `-force-symbolic`.
- `https://aarneranta.github.io/mathlib-informath/`

Open problems

Open problems: coverage

- Scale. 705 of Mathlib's 240,000 declarations on the pages; 26,679 generated entries for its definitions, unreviewed: composite names (`XEquivY`), bundled maps without arguments ("the dual"), verbs (`PreservesX`), 2,000 unknown abbreviations; the words of the other languages in `translations.tsv` to be reviewed.
- Tactic proofs. 242 of the 620 theorems here, and most of Mathlib's, are proved `by` tactics: 25 have proof texts so far, in English; the others, the other languages, and the term and tactic proofs to be told apart on the page. Long `simp_rw` goals, `by_contra` as "it is enough to show that we have a contradiction".
- Unworded statements (131 of 705):
  - multi-letter variables (`lt`, `self`, `motive`) are shown as lt and counted as constants;
  - function arguments: `Set.ofPred`, the motives of induction;
  - `Leq` used unapplied: $\mathrm{Leq}(x_2, x_3)$.
- Decidability instances defined as declarations are still exported; two `Decidable` proofs (with `dite`) have no proof text.
- Theorem labels are mangled names: `Nat__Prime__two_le`.

Open problems: language

- Proof texts where a lemma or constructor is used unapplied: “For all sets x and functions r ... and elements irrefl of Proof applied to the proposition that ..., r is irreflexive.” Bound variables are renumbered (x_4, x_5) where the source reuses names.
- Meaning that depends on the type: $\leq$ on finite sets reads “less than or equal to”, where Mathlib means “subset of”.
- French: “pour tous des nombres naturels”, “il n’est pas le cas que” (a calque of “it is not the case that”), “une relation d’équivalences”; three French modules are partial.
- Czech: unreviewed words; “existuje pirozené číslo p, takový, e” (takové): “such that” agrees only after a CN.
- Informath:
  - `$` in a quoted verbal form splits a table entry (`splitEntry`), which rules out binder shapes in tables;
  - a proposition inside a macro argument loses its notation;
  - “does not have a upper bound”;
  - `proof2dedukti` does not cover proofs, so the back-translation of `-json` crashes on them.
- German: GF’s linearization is exponential in nested noun phrases (48 statements over the limit); “elemente”; “Dann $2 \leq$ p”.
- Swedish: “lika till” (lika med) and Czech “rovná do”: `Eqt`’s “is equal to” parses word by word.

Summary

Who did what

Already in Informath	From my prompts	Added by Claude
The GF grammar and its RGL-based concrete syntaxes, French among them (2025–; `WikidataWordsFre` completed with Claude in July)	The goal: an informalization system for Mathlib with Informath	`Export.lean`: the translation rules, classes as predicates, `__app` coercions, eta-expanded binders, proofs in `nd.dk` rules, filters
The Dedukti dialect, its base constants, symbol tables with verbal, symbolic and raw macro forms	The choices among Claude's options: own exporter, Nat primes first, a new project with the cache	`mapping.tsv`; `mathlib.dkgf` (108 entries) with `suggest-entries.py`
NLG with variants and ranking; `-to-symbolic-latex`, `-force-symbolic`, `-parse-example`	The page's contents; GitHub hosting (I created the repository and pushed)	`build-index.py`, `build-html.py`: index, pages, macros, proof links
`-proof-text` (since June, 8 of 21 commits with Claude), and godement-index's way of showing proofs	A page per part; keeping the definientia and showing the proofs; words from the sources; new words in `Mathlib.gf`	Words in `Mathlib.gf`, `MathlibFre.gf`, and partial French modules
The regression suite, the autoformalization template	The next modules; French as a third row; one index loading the grammar once; Czech, hidden; hypotheses aggregated, reordered; German, with an aggressive time limit; tactic proofs as proof units	Informath: `abstractionTypes`, `-languages`, multilingual proof texts, `-no-back-to-dedukti`, `ProofUnitsFre`, Czech, `aggregateHypos`, `-tactic-proof`; `export_-tactics`

`github.com/aarneranta/mathlib-informath`

705 Mathlib statements, 265 proofs, in English, French, Czech, German, Swedish and LaTeX; all of Mathlib's 38,667 definitions read in English and as formulas